

Azure File Storage Job 模式与权限设计

Windows Service 迁移方案 • 文件操作效率分析 • 权限访问架构

业务场景	Windows Service ██████ -> Azure Job
核心问题1	██████/██████████████████
核心问题2	█ Branch/SealType ████████████████
文档日期	2026-04-15

目录

1. 业务场景回顾与问题定义
2. 文件操作效率分析
3. 推荐方案：Azure Job 架构设计
4. Azure Functions 实现代码
5. 权限访问设计方案
6. SAS 令牌权限控制详解
7. 完整权限架构示例
8. ASP.NET MVC 权限集成

1. 业务场景回顾与问题定义

1.1 当前架构

Windows Service 定时扫描目录，按 Branch 和 SealType 重新分类文件。

步骤	操作	说明
1	扫描目录	遍历 RequestID 文件夹下的所有文件
2	读取文件属性	获取文件类型、大小、创建时间
3	确定目标路径	根据 Branch/SealType 生成新路径
4	创建目录	必要时创建目标目录结构
5	移动文件	File.Move() 到新路径
6	更新记录	可选：更新数据库分类状态

1.2 核心问题

问题A：效率担忧 - Azure 文件操作是否足够快？

问题B：权限访问 - 如何让不同用户访问不同文件夹层级？

2. 文件操作效率分析

2.1 Azure File Storage 操作限制

操作类型	单次延迟	每秒限制	大批量建议
创建目录	3-5 ms	2000 次/秒	批量时每批 100-500 个
删除目录	5-10 ms	500 次/秒	需先清空内容
列举文件	10-50 ms	200 次/秒	每页最大 5000 条
文件上传	5-20 MB/s	取决于大小	大文件用分块上传
文件移动	50-100 ms	10-50 次/秒	需下载+上传，非原地移动

重要：Azure File Storage 不支持原地的 File.Move()！移动文件实际上是【下载到内存】->【上传到新路径】->【删除原文件】三个操作。

2.2 性能瓶颈对比

场景	本地 NTFS	Azure File	差异
扫描 1000 个文件	1-2 秒	30-60 秒	网络延迟累积
移动文件	即时	50-100 ms	需下载再上传
创建 100 个目录	10-50 ms	500-1000 ms	API 调用开销

2.3 大规模操作优化策略

策略	说明	效果提升
并行处理	使用多线程/多实例并发操作	3-5 倍
异步队列	将任务放入队列异步处理	吞吐量提升 10 倍
增量扫描	只扫描新增文件，不全量扫描	减少 80%
分片处理	按日期/类型分片，错峰处理	避免限流

3. 推荐方案：Azure Job 架构设计

推荐方案：使用 Azure Functions (Timer Trigger) + Azure Storage SDK 替代 Windows Service，实现云端 Job 模式。

3.1 三种 Job 实现方案对比

方案	Azure Functions	Azure Logic Apps	Azure Batch
适用场景	轻量级定时任务	可视化流程编排	大规模计算密集型
成本	免费额度充足	按操作计费	按计算时计费
本场景推荐	5星	3星	2星
推荐原因	简单、轻量、可扩展	适合复杂流程	过度设计

3.2 Azure Functions Job 架构

组件	技术选型	职责
触发器	Timer Trigger	定时触发扫描任务（如每 5 分钟）
扫描逻辑	File Share SDK	遍历待处理目录，筛选未分类文件
分类规则	配置表/数据库	Branch + SealType -> 目标路径映射
文件操作	Azure File SDK	下载 -> 上传到新路径 -> 删除原文件
状态记录	Azure Queue/Table	记录处理进度，支持断点续传

3.3 增量处理流程

阶段	操作	说明
1. 检查状态表	Query Table Storage	获取上次处理到的最新 RequestID
2. 扫描增量	List Files (marker)	只扫描比上次更新的目录
3. 分类处理	Foreach + 异步	对每个文件执行分类逻辑
4. 更新状态	Update Table	记录当前处理位置（断点续传）

4. Azure Functions 实现代码

4.1 Timer Trigger Function (主 Job)

```
using System; using System.IO; using System.Threading.Tasks; using Microsoft.Azure.WebJobs; using
Microsoft.Extensions.Logging; using Azure.Storage.Files.Shares; public static class
FileClassificationJob { [FunctionName("FileClassificationJob")] public static async Task Run(
[TimerTrigger("0 */5 * * * *")] TimerInfo myTimer, // 5分钟 ILoggger log) { log.LogInformation($"Job
开始: {DateTime.Now}"); // 1. Azure File Share var connectionString =
Environment.GetEnvironmentVariable("AzureStorage"); var shareClient = new
ShareClient(connectionString, "fileshare"); // 2. var pendingDirs = await
GetPendingDirectoriesAsync(shareClient, log); foreach (var requestId in pendingDirs) { try { await
ProcessRequestDirectoryAsync(shareClient, requestId, log); await
UpdateProcessingStatusAsync(requestId, "Completed", log); } catch (Exception ex) { log.LogError($"
{requestId} : {ex.Message}"); } } } private static async Task ProcessRequestDirectoryAsync(
ShareClient shareClient, string requestId, ILogger log) { var requestDir =
shareClient.GetDirectoryClient(requestId); // RequestID await foreach (ShareFileItem
fileItem in requestDir.GetFilesAndDirectoriesAsync()) { if (!fileItem.IsDirectory) { await
ClassifyFileAsync(shareClient, requestId, fileItem.Name, log); } } } private static async Task
ClassifyFileAsync( ShareClient shareClient, string requestId, string fileName, ILogger log) { //
Branch SealType var (branch, sealType) = DetermineClassification(fileName); // var
targetDirPath = $"{branch}/{sealType}/{requestId}"; var targetDir =
shareClient.GetDirectoryClient(targetDir); await targetDir.CreateIfNotExistsAsync(); // var
sourceFile = shareClient.GetDirectoryClient(requestId).GetFileClient(fileName); var targetFile =
targetDir.GetFileClient(fileName); // -> -> var download = await
sourceFile.DownloadAsync(); using var stream = new MemoryStream(); await
download.Value.Content.CopyToAsync(stream); stream.Position = 0; await
targetFile.CreateAsync(stream.Length); await targetFile.UploadRangeAsync(new HttpRange(0,
stream.Length), stream); await sourceFile.DeleteAsync(); log.LogInformation($" {fileName}
{targetDirPath}"); } private static (string branch, string sealType) DetermineClassification(string
fileName) { // return ("company", "beijing"); }
```

4.2 性能优化：并行处理

```
// SemaphoreSlim private static readonly SemaphoreSlim Semaphore = new SemaphoreSlim(10);
private static async Task ProcessWithThrottlingAsync( List items, Func processFunc, ILogger log) { var
tasks = items.Select(async item => { await Semaphore.WaitAsync(); try { await processFunc(item); }
finally { Semaphore.Release(); } }); await Task.WhenAll(tasks); }
```

5. 权限访问设计方案

5.1 权限需求示例

用户角色	可访问路径	说明
系统管理员	fileshare/*	所有文件夹完全访问
北京分公司用户	company/beijing/*	访问北京公司所有分类
广州分公司用户	contract/guangzhou/*	访问广州合同分类
普通员工	company/beijing/{RequestID}/*	只能看自己的请求

5.2 三种权限控制方案对比

方案	实现方式	优点	缺点	推荐度
存储账户密钥	直接挂载 SMB	简单、无限权限	权限过大、不安全	仅开发测试
SAS 令牌	生成带权限的 URL	细粒度、可控过期	URL 管理复杂	5星推荐
Azure AD RBAC	Azure 文件共享 RBAC	统一身份管理	配置复杂、需 AAD	4星

推荐方案：SAS 令牌（Shared Access Signature）为不同用户生成不同权限级别的 SAS URL，实现细粒度访问控制。

6. SAS 令牌权限控制详解

6.1 为不同用户生成不同路径权限的 SAS

[illegible]

6.2 不同角色的 SAS 权限示例

角色	SAS 权限范围	有效时间	可执行操作
北京员工	fileshare/company/beijing/*	8 小时	读取、下载、列举
广州员工	fileshare/contract/guangzhou/*	8 小时	读取、下载、列举
上传用户	fileshare/uploads/*	24 小时	写入、上传、创建目录
管理员	fileshare/*	1 小时	全部操作
临时访客	fileshare/shared/temp/*	30 分钟	读取

7. 完整权限架构示例

7.1 基于角色的权限映射表

角色ID	角色名称	可访问路径模式	Branch	SealType	权限
R001	北京-管理员	company/beijing/*	company	beijing	RLWD
R002	北京-普通	company/beijing/{RequestID}/*	company	beijing	RL
R003	广州-管理员	contract/guangzhou/*	contract	guangzhou	RLWD
R004	广州-普通	contract/guangzhou/{RequestID}/*	contract	guangzhou	RL
R005	审计员	company/*	company	*	RL
R006	超级管理员	*	*	*	RLWD

R=Read, L=List, W=Write, D=Delete

7.2 权限检查服务实现

```
public class AzureFileAccessService { /// ██████████ SAS URL public string
GetAccessibleSasUrl(string userRole, string requestId) { var path = GetRoleAccessiblePath(userRole,
requestId); var permissions = GetRolePermissions(userRole); var validDuration =
GetRoleValidDuration(userRole); return GenerateSasUrl(path, permissions, validDuration); } ///
██████████ public bool CanAccess(string userRole, string filePath) { var allowedPaths =
GetAllowedPaths(userRole); return allowedPaths.Any(p => MatchesPathPattern(filePath, p)); } ///
██████████ SAS ███ public async Task< GetUserDataFilesAsync( string userRole, string
basePath) { var sasUrl = GetAccessibleSasUrl(userRole, null); var shareClient = new ShareClient(new
Uri(sasUrl)); var files = new List<>(); var directoryClient = shareClient.GetDirectoryClient(basePath);
await foreach (ShareFileItem item in directoryClient.GetFilesAndDirectoriesAsync()) { files.Add(new
AzureFileInfo { Name = item.Name, IsDirectory = item.IsDirectory, FileUrl =
$"{sasUrl.Split('?')[0]}/{item.Name}" }); } return files; } }
```

8. ASP.NET MVC 权限集成

8.1 Controller 集成 SAS 访问

8.2 前端使用 SAS URL 访问

```
// 1.1.1.1 SAS URL 1.1.1.1 async function loadFolder(folderPath) { const response = await
fetch(`/FileShare/GetFolderSasUrl?folder=${folderPath}`); const { sasUrl } = await response.json(); //
1.1.1.1.1 window.open(`${sasUrl}/myfile.pdf`, '_blank'); // 1.1.1.1.2 REST API 1.1.1.1.2 const files =
await listFilesWithSas(sasUrl); } // REST API 1.1.1.1.2 async function listFilesWithSas(sasUrl) { const
apiUrl = `${sasUrl}&restype=directory&list`; const response = await fetch(apiUrl); const text =
await response.text(); const parser = new DOMParser(); const xml = parser.parseFromString(text,
'text/xml'); const entries = xml.querySelectorAll('File'); return Array.from(entries).map(entry => ({
name: entry.querySelector('Name').textContent, size: entry.querySelector('Content-Length').textContent
})); }
```

总结: 1. Windows Service 迁移到 Azure Functions Job, 推荐 Timer Trigger + SDK 批量操作 2. 文件操作瓶颈在于网络延迟, 通过并行 + 增量处理可有效优化 3. 权限控制推荐使用 SAS 令牌, 为不同角色生成不同路径权限的 SAS URL 4. ASP.NET MVC 通过 IAzureFileAccessService 统一管理权限